PROJET PERSONNEL — MASTER 1 INFORMATIQUE & MATHÉMATIQUES APPLIQUÉES
2025/2026

Réseaux de Neurones Graphiques

pour la Modélisation par Approximation des Écoulements de Fluides Incompressibles

— Rapport de Projet M1 —

Mots-clés : Réseaux de neurones graphiques (GNN) · Physique a priori · Mécanique des fluides computationnelle · Simulation par substitution · GNS · MeshGraphNet · E(3)-GNN · GMN

Auteur : **AHNANI Ali**

Type de projet :	Recherche appliquée — Apprentissage pour les sciences
Source de données :	Polymathic AI — The WELL (`polymathic-ai.org/the_well`)
Domaine :	IA pour les Sciences Physiques (*AI4Science*)
Prérequis :	Python · Deep Learning · GNN ou CFD
Volume de travail :	60–80 heures
Fichiers fournis :	Notebook Jupyter + 7 modules Python

"Physics is the universe's operating system. Graph networks are a natural language to speak it."

(Paraphrase de Peter Battaglia, DeepMind)

0 Contents

1 Introduction et Contexte Général **3**
1.1 Le Défi de la Simulation Numérique en Mécanique des Fluides . . . 3
1.2 Les Modèles de Substitution par Apprentissage Machine . . . 3
1.3 Pourquoi les Graphes ? . . . 4

2 Fondements Théoriques **5**
2.1 Équations de Navier-Stokes pour les Fluides Incompressibles . . . 5
2.1.1 La Contrainte d'Incompressibilité . . . 5
2.1.2 Le Nombre de Reynolds et les Régimes d'Écoulement . . . 5
2.2 L'Instabilité de Kelvin-Helmholtz . . . 6
2.3 Réseaux de Neurones Graphiques . . . 7
2.3.1 Représentation par Graphe . . . 7
2.3.2 Message Passing Neural Networks (MPNN) . . . 7
2.3.3 Architecture Encode-Process-Decode . . . 7
2.4 Équivariance et Symétries Physiques . . . 8

3 Le Dataset : The WELL de Polymathic AI **9**
3.1 Polymathic AI et la Vision des Fondations Physiques . . . 9
3.2 Description du Dataset `shear_flow` . . . 9
3.2.1 Structure des Fichiers HDF5 . . . 9
3.3 Pré-traitement et Construction du Graphe . . . 10

4 Architectures à Implémenter **11**
4.1 GNS / MeshGraphNet — Simulateur sur Graphe . . . 11
4.1.1 Principes Fondamentaux . . . 11
4.1.2 Couche d'Interaction (InteractionNetworkLayer) . . . 11
4.1.3 Différences GNS vs MeshGraphNet . . . 11
4.2 E(3)-GNN — Réseau Équivariant . . . 11
4.2.1 Motivation : Briser l'Équivariance Coûte des Données . . . 11
4.2.2 Équations de Mise à Jour EGNN . . . 12
4.3 GMN — Graph Mechanics Network . . . 12
4.3.1 Architecture Complète et Contraintes Physiques . . . 12
4.3.2 La Décomposition de Helmholtz-Hodge . . . 12

5 Méthodologie Expérimentale **14**
5.1 Pipeline d'Entraînement . . . 14
5.1.1 Schéma Général . . . 14
5.1.2 Bruit de Marche Aléatoire . . . 14
5.1.3 Rollout Autoregressif pour l'Évaluation . . . 14
5.2 Configuration des Expériences . . . 14

6 Résultats Expérimentaux **15**
6.1 Résultats Quantitatifs (CSV) . . . 15
6.2 Courbes d'Apprentissage . . . 15
6.3 Métriques de Rollout . . . 16
6.3.1 NMSE Normalisée . . . 16
6.3.2 Temps de Validité t^* . . . 16
6.3.3 Spectre d'Énergie Cinétique . . . 16

7 Organisation du Code et des Fichiers **19**
7.1 Description des Modules Python . . . 19
7.1.1 `src/models/meshgraphnet.py` . . . 19
7.1.2 `src/models/e3gnn.py` . . . 19
7.1.3 `src/models/gmn.py` . . . 19
7.1.4 `src/data/preprocessing.py` . . . 20
7.1.5 `src/utils/metrics.py` . . . 20

8 Livrables, Calendrier et Critères d'Évaluation **21**
8.1 Livrables . . . 21
8.2 Calendrier sur 14 Semaines . . . 21
8.3 Critères d'Évaluation . . . 21
8.3.1 Pistes d'Extension (Bonus) . . . 21

9 Références **22**

A Annexe A : Rappels Mathématiques **23**
A.1 Transformée de Fourier Discrète 2D . . . 23
A.2 Critère Q pour la Détection des Tourbillons . . . 23

B Annexe B : Ressources et Liens Utiles **23**

C Annexe C : Fichiers Python Fournis **24**

1 Introduction et Contexte Général

1.1 Le Défi de la Simulation Numérique en Mécanique des Fluides

La simulation numérique des écoulements de fluides est au cœur de certains des problèmes les plus importants et complexes de la science et de l'ingénierie moderne : la prévision météorologique, la conception aérodynamique des avions, la modélisation des courants océaniques, la prédiction des phénomènes climatiques, ou encore la conception de réacteurs chimiques et nucléaires.

Les méthodes classiques de la **mécanique des fluides computationnelle** (*Computational Fluid Dynamics*, CFD) — volumes finis, éléments finis, différences finies — permettent d'obtenir des solutions extrêmement précises aux équations de Navier-Stokes. Cependant, leur coût calculatoire est prohibitif : une simulation de turbulence atmosphérique à haute résolution peut nécessiter des millions d'heures de calcul CPU, même sur les supercalculateurs modernes.

Cette contrainte de coût est un frein majeur à de nombreuses applications :

- **Optimisation** : Évaluer des milliers de configurations (forme d'aile, profil de pale d'éolienne) nécessiterait autant de simulations CFD complètes.
- **Assimilation de données** : Les méthodes d'ensemble en météorologie requièrent des centaines de simulations parallèles pour estimer les incertitudes.
- **Contrôle en temps réel** : Les applications industrielles nécessitent des prédictions en quelques millisecondes, incompatibles avec les temps de simulation classiques.

1.2 Les Modèles de Substitution par Apprentissage Machine

Face à ce défi, une approche émergente et prometteuse consiste à construire des **modèles de substitution** (*surrogate models* ou *emulators*) basés sur l'apprentissage automatique. Le principe est le suivant :

1. On génère un ensemble de données d'entraînement en exécutant des simulations CFD de haute qualité (coûteuses, mais réalisées une seule fois).
2. On entraîne un modèle de machine learning pour apprendre la dynamique du système à partir de ces données.
3. Le modèle entraîné peut ensuite prédire de nouvelles trajectoires en une fraction du temps de la simulation complète.

Les approches classiques incluent les réseaux de neurones convolutifs (CNN) sur grilles régulières et les méthodes basées sur les opérateurs neuronaux (Fourier Neural Operators, DeepONet). Cependant, ces approches souffrent de limitations importantes : elles sont liées à une résolution fixe de grille et ne peuvent pas gérer facilement des géométries complexes ou des maillages irréguliers.

Objectifs du Projet

Ce projet explore les **réseaux de neurones graphiques (GNN)** comme alternative flexible et physiquement motivée pour l'émulation de simulations de fluides. L'objectif central est d'implémenter, entraîner et comparer rigoureusement trois architectures — GNS/MeshGraphNet, E(3)-GNN et GMN — sur les données *The WELL* de Polymathic AI, et d'évaluer leur capacité à prédire des écoulements sur de longs horizons temporels.

1.3 Pourquoi les Graphes ?

Un maillage CFD est naturellement un graphe : les cellules sont les nœuds, les interfaces entre cellules sont les arêtes. Cette représentation offre plusieurs avantages fondamentaux par rapport aux CNN :

- **Géométrie arbitraire** : Les graphes s'adaptent nativement aux géométries complexes (ailes d'avion, conduits, etc.) sans imposer de discrétisation en grille régulière.
- **Invariance par permutation** : L'ordre des nœuds n'importe pas, ce qui est physiquement cohérent.
- **Inductive biases** : Il est possible d'encoder des connaissances physiques (symétries, conservation de la masse) directement dans l'architecture.
- **Passage à l'échelle** : Les GNN peuvent s'adapter à des résolutions variables lors de l'inférence (sans réentraînement).

2 Fondements Théoriques

2.1 Équations de Navier-Stokes pour les Fluides Incompressibles

Les écoulements de fluides visqueux incompressibles newtoniens sont régis par les équations de **Navier-Stokes** :

$$\underbrace{\rho\left(\frac{\partial \boldsymbol{u}}{\partial t} + (\boldsymbol{u}\cdot\nabla)\boldsymbol{u}\right)}_{\text{inertie}} = \underbrace{-\nabla p}_{\text{gradient de pression}} + \underbrace{\mu\,\Delta\boldsymbol{u}}_{\text{diffusion visqueuse}} + \underbrace{\rho\,\boldsymbol{f}}_{\text{forçage ext.}} \tag{1}$$

$$\underbrace{\nabla\cdot\boldsymbol{u} = 0}_{\text{incompressibilité}} \tag{2}$$

où $\boldsymbol{u}(\boldsymbol{x},t) \in \mathbb{R}^d$ est le champ de vitesse, $p(\boldsymbol{x},t)$ la pression, ρ la masse volumique (constante pour un fluide incompressible), μ la viscosité dynamique, et $\boldsymbol{f}$ les forces extérieures.

2.1.1 La Contrainte d'Incompressibilité

L'équation (2) est la traduction mathématique de la *conservation de la masse* pour un fluide dont la densité ne varie pas. Elle implique que le champ de vitesse est à **divergence nulle** : il n'existe pas de sources ni de puits dans l'écoulement. Cette contrainte couple implicitement la vitesse et la pression, et est à l'origine de la principale difficulté numérique de la résolution des équations de Navier-Stokes. En particulier, elle interdit toute prédiction de vitesse dont le champ associé ne serait pas solénoïdal.

Insight clé : Contrainte d'incompressibilité et GNN

Un GNN naïf prédit des incréments de vitesse sans garantir $\nabla\cdot\boldsymbol{u} = 0$. Cette violation de la contrainte physique se traduit par des instabilités croissantes lors du rollout : les erreurs s'accumulent et le simulateur diverge. Le module GMN (Section 4.3) adresse directement ce problème via une projection spectrale.

2.1.2 Le Nombre de Reynolds et les Régimes d'Écoulement

Le **nombre de Reynolds** est le paramètre adimensionnel fondamental qui caractérise le régime d'un écoulement :

$$Re = \frac{\rho\,U\,L}{\mu} = \frac{U\,L}{\nu} \tag{3}$$

où U est une vitesse caractéristique, L une longueur caractéristique, et $\nu = \mu/\rho$ la viscosité cinématique. Il représente le rapport des forces d'inertie aux forces visqueuses. Pour $Re \ll 1$ (faible vitesse, fluide visqueux), l'écoulement est *laminaire* et prévisible. Pour $Re \gg 1$, il bascule vers la *turbulence* : un régime chaotique, multiéchelle, extrêmement sensible aux conditions initiales — et donc particulièrement difficile à émuler sur de longs horizons.

L'écoulement de cisaillement étudié dans ce projet se situe dans un régime de Re modéré à élevé ($Re \in [500, 5000]$), suffisamment riche pour présenter des structures cohérentes intéressantes tout en restant relativement prévisible à moyen terme.

2.2 L'Instabilité de Kelvin-Helmholtz

L'instabilité de Kelvin-Helmholtz (KH) est un phénomène fondamental de la mécanique des fluides : elle apparaît à l'interface entre deux couches de fluide se déplaçant à des vitesses différentes (cisaillement). Dans notre configuration 2D périodique :

$$u_x(y, t = 0) = \tanh\left(\frac{y - L/2}{\delta}\right) + \epsilon\, \eta(x, y), \qquad u_y(y, t = 0) = \epsilon' \sin(x) \tag{4}$$

où δ est l'épaisseur de la couche de cisaillement, ϵ, ϵ' sont les amplitudes de la perturbation initiale aléatoire $\eta \sim \mathcal{N}(0, 1)$.

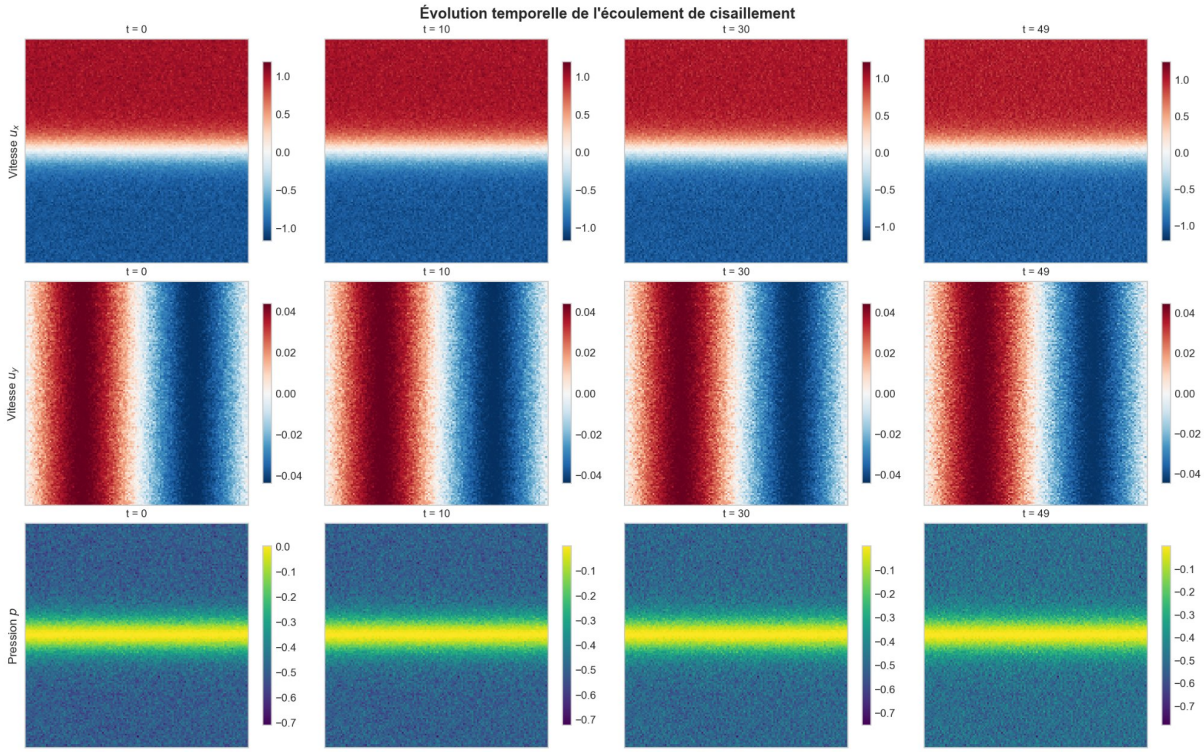


Figure 1: **Évolution temporelle de l'écoulement de cisaillement** : champs de vitesse u_x, u_y et pression p à quatre instants représentatifs ($t = 0, 10, 30, 49$). À $t = 0$, le profil de cisaillement est clairement visible (rouge/bleu pour les courants opposés). La perturbation initiale aléatoire déclenche l'instabilité de Kelvin-Helmholtz qui évolue progressivement vers un régime complexe. La reproduction fidèle de cette dynamique multi-échelles par le GNN est l'objectif central du projet.

Comme illustré à la Figure 1, l'évolution est qualitativement riche : les structures tourbillonnaires (régions de vorticite intense) se forment, grossissent par coalescence et finissent par dominer l'écoulement. Ce processus est typiquement observé en géophysique (jet-stream atmosphérique, courants marins) et en astrophysique.

2.3 Réseaux de Neurones Graphiques

2.3.1 Représentation par Graphe

Un domaine physique discrétisé est représenté comme un graphe $\mathcal{G} = (\mathcal{V}, \mathcal{E})$:

- **Nœuds** $\mathcal{V}$: cellules du maillage, portant les variables d'état (u_x, u_y, p).
- **Arêtes** $\mathcal{E}$: connectivité du maillage (interfaces entre cellules), portant les attributs géométriques $(\Delta x, \Delta y, \|\Delta \boldsymbol{r}\|)$.

La construction du graphe se fait en reliant chaque nœud à ses voisins immédiats (graphe 4-connexe) ou à tous les nœuds dans un rayon r (graphe à rayon), avec conditions aux limites périodiques.

2.3.2 Message Passing Neural Networks (MPNN)

Le paradigme central de tous les GNN est le **passage de messages**. À chaque couche l, les représentations des nœuds sont mises à jour en deux étapes :

Étape 1 — Messages : Chaque arête (i, j) produit un message en combinant les représentations des nœuds connectés et les attributs de l'arête :

$$\boldsymbol{m}_{ij}^{(l)} = \psi^{(l)}\left(\boldsymbol{h}_i^{(l)}, \boldsymbol{h}_j^{(l)}, \boldsymbol{e}_{ij}\right) \tag{5}$$

Étape 2 — Agrégation et mise à jour : Chaque nœud agrège les messages entrants et met à jour sa représentation :

$$\boldsymbol{h}_i^{(l+1)} = \phi^{(l)}\left(\boldsymbol{h}_i^{(l)}, \bigoplus_{j \in \mathcal{N}(i)} \boldsymbol{m}_{ij}^{(l)}\right) \tag{6}$$

où $\bigoplus$ est une opération d'agrégation permutation-invariante (somme, moyenne, max), ψ et ϕ sont des MLP appris. Après L couches, chaque représentation $\boldsymbol{h}_i^{(L)}$ encode l'information dans un voisinage de rayon L autour du nœud i.

2.3.3 Architecture Encode-Process-Decode

Les simulateurs GNN suivent invariablement l'architecture **Encode-Process-Decode** (Battaglia et al., 2018) :

- **Encodeur** : Transforme les variables physiques brutes (u_x, u_y, p) en représentations latentes de dimension fixe d_{hidden}.
- **Processeur** : L couches de passage de messages qui propagent l'information dans le graphe.
- **Décodeur** : Extrait la quantité d'intérêt (accélération, incrément de vitesse) depuis la représentation latente finale.

2.4 Équivariance et Symétries Physiques

Un aspect crucial de l'architecture des GNN pour la physique est la possibilité d'intégrer des **symétries** directement dans le modèle. Pour les équations de Navier-Stokes, les symétries fondamentales sont :

Symétrie	**Transformation**	**Implication**
Translation spatiale	$\boldsymbol{x} \to \boldsymbol{x} + \boldsymbol{c}$	Invariance de la physique locale
Rotation	$\boldsymbol{u} \to R\boldsymbol{u}$	Équivariance des champs vectoriels
Symétrie temporelle	$t \to t + \tau$	Stationnarité (fluide homogène)
Réflexion	$x \to -x$	Équivariance (pas de chiralité)

Pourquoi les symétries améliorent la généralisation ? Un modèle non-équivariant doit *apprendre* les symétries depuis les données. Cela consomme de la capacité du modèle et nécessite plus de données d'entraînement. Un modèle équivariant *garantit* ces symétries par construction, ce qui réduit l'espace d'hypothèses et améliore la généralisation aux nouvelles conditions initiales et orientations.

3 Le Dataset : The WELL de Polymathic AI

3.1 Polymathic AI et la Vision des Fondations Physiques

Polymathic AI est une initiative scientifique collaborative fondée en 2022 regroupant des chercheurs du Flatiron Institute (Simons Foundation), NYU, Princeton, Columbia et d'autres institutions. Sa mission est de développer des **modèles de fondation** (*foundation models*) pour les sciences physiques.

The WELL (*Well-curated scientific simulations for Emulation, Learning and Large foundation model pretraining*) est leur jeu de données phare : une collection de ≈ 15 To de simulations physiques de haute qualité, publiée en open access en 2024 (arXiv:2412.00568). Elle couvre 16 phénomènes différents allant de la turbulence 2D à la convection stellaire.

3.2 Description du Dataset `shear_flow`

Pour ce projet, le dataset principal est `shear_flow` qui simule l'instabilité de Kelvin-Helmholtz dans un domaine 2D périodique, résolu par une méthode pseudo-spectrale de haute précision.

Paramètre	Symbole	Plage / Valeur	Distribution
Résolution grille	$N_x \times N_y$	128^2 – 512^2	Fixe par split
Pas de temps CFD	Δt	10^{-3}	Fixe
Pas de temps ML	Δt_{ML}	10^{-2}	Sous-échantillonné
Épaisseur cisaillement	δ	$[0.1, 0.5]$	Uniforme
Amplitude perturbation	ϵ	$[0.01, 0.1]$	Log-uniforme
Nombre de Reynolds	Re	$[500, 5000]$	Log-uniforme
Nombre de trajectoires	N_{traj}	≈ 1000	–
Longueur trajectoires	T	200	–

3.2.1 Structure des Fichiers HDF5

Les données sont stockées au format `HDF5`, un format binaire hiérarchique conçu pour les données scientifiques volumineuses. Chaque fichier contient :

```python
import h5py
with h5py.File('shear_flow_train.hdf5', 'r') as f:
    # Champs principaux : shape (N_traj, T, Nx, Ny)
    vx = f['velocity_x']    # Vitesse horizontale u_x
    vy = f['velocity_y']    # Vitesse verticale  u_y
    p  = f['pressure']      # Pression p
    # Meta-donnees par trajectoire
    Re_vals = f['metadata/reynolds_number'][:]    # (N_traj,)
    eps_vals = f['metadata/perturbation_amp'][:] # (N_traj,)
```

Listing 1: Exploration de la structure HDF5 (The WELL)

3.3 Pré-traitement et Construction du Graphe

La conversion des données grille en graphe PyTorch Geometric se déroule en trois étapes :

Étape 1 — Normalisation z-score :

$$\tilde{\phi} = \frac{\phi - \mu_{\phi,\text{train}}}{\sigma_{\phi,\text{train}}} \tag{7}$$

Cette normalisation est calculée *uniquement* sur l'ensemble d'entraînement pour éviter toute fuite de données (*data leakage*).

Étape 2 — Construction des arêtes : Pour une grille $N_x \times N_y$ avec conditions périodiques, le graphe 4-connexe comporte $4 \cdot N_x \cdot N_y$ arêtes. Chaque arête (i, j) porte comme attribut $\boldsymbol{e}_{ij} = (\Delta x/N_x,\ \Delta y/N_y,\ \|(\Delta x, \Delta y)\|)$.

Étape 3 — Bruit de marche aléatoire : Pour améliorer la robustesse du modèle aux erreurs d'accumulation lors du rollout, on ajoute un bruit d'entraînement (voir Section 5.1.2).

4 Architectures à Implémenter

4.1 GNS / MeshGraphNet — Simulateur sur Graphe

4.1.1 Principes Fondamentaux

GNS (*Graph Network-based Simulator*, Sanchez-Gonzalez et al., ICML 2020) et **MeshGraphNet** (Pfaff et al., NeurIPS 2021) sont les architectures de référence dans le domaine. Elles suivent l'architecture Encode-Process-Decode avec une caractéristique clé : la prédiction d'une **accélération** plutôt que de l'état suivant directement.

Formellement, à chaque pas de temps :

$$\hat{\boldsymbol{a}}_t = f_\theta(\mathcal{G}_t) \quad \text{[prédiction GNN]} \tag{8}$$

$$\hat{\boldsymbol{u}}_{t+1} = \boldsymbol{u}_t + \hat{\boldsymbol{a}}_t \cdot \Delta t \quad \text{[intégration Euler]} \tag{9}$$

4.1.2 Couche d'Interaction (InteractionNetworkLayer)

La couche centrale effectue une mise à jour conjointe des arêtes et des nœuds avec connexions résiduelles :

$$\boldsymbol{e}'_{ij} = \boldsymbol{e}_{ij} + \phi_e(\boldsymbol{h}_i, \boldsymbol{h}_j, \boldsymbol{e}_{ij}) \quad \text{[arêtes]} \tag{10}$$

$$\boldsymbol{h}'_i = \boldsymbol{h}_i + \phi_v\left(\boldsymbol{h}_i, \textstyle\sum_{j\in\mathcal{N}(i)} \boldsymbol{e}'_{ij}\right) \quad \text{[nœuds]} \tag{11}$$

```python
class InteractionNetworkLayer(MessagePassing):
    def forward(self, x, edge_index, edge_attr):
        src, dst = edge_index
        # Mise a jour des aretes (phi_e)
        edge_in = torch.cat([x[src], x[dst], edge_attr], dim=-1)
        new_edge = edge_attr + self.edge_mlp(edge_in)  # residu
        # Agregation et mise a jour des noeuds (phi_v)
        agg = self.propagate(edge_index, x=x, edge_attr=new_edge)
        new_x = x + self.node_mlp(torch.cat([x, agg], dim=-1))  # residu
        return new_x, new_edge
```

Listing 2: Couche InteractionNetworkLayer (meshgraphnet.py)

4.1.3 Différences GNS vs MeshGraphNet

Aspect	GNS	MeshGraphNet
Type de maillage	Particules / grilles régulières	Maillages non-structurés
Graphe monde	Non	Optionnel (longue portée)
Types de nœuds	Homogène	Intérieur / Frontière
Attributs arêtes	Déplacements relatifs	Géométrie maillage

4.2 E(3)-GNN — Réseau Équivariant

4.2.1 Motivation : Briser l'Équivariance Coûte des Données

Les **E(3)-GNN** (Satorras et al., ICML 2021) résolvent ce problème en garantissant l'équivariance par construction. L'idée centrale d'EGNN est de maintenir deux types de représentations :

- $\boldsymbol{h}_i \in \mathbb{R}^d$: **features invariantes** (scalaires) — ne changent pas sous rotation/translation.
- $\boldsymbol{x}_i \in \mathbb{R}^n$: **coordonnées équivariantes** (vecteurs) — se transforment comme des vecteurs sous rotation.

4.2.2 Équations de Mise à Jour EGNN

$$\boldsymbol{m}_{ij} = \phi_e\big(\boldsymbol{h}_i,\, \boldsymbol{h}_j,\, \|\boldsymbol{r}_{ij}\|^2\big) \quad \text{[message invariant]} \tag{12}$$

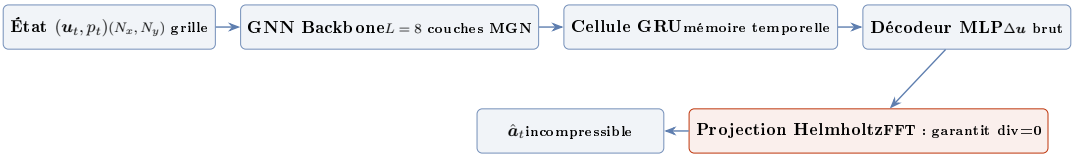
$$\boldsymbol{h}'_i = \phi_h\Big(\boldsymbol{h}_i,\, \textstyle\sum_{j\neq i} \boldsymbol{m}_{ij}\Big) \quad \text{[mise à jour invariante]} \tag{13}$$

$$\boldsymbol{x}'_i = \boldsymbol{x}_i + C\sum_{j\neq i}(\boldsymbol{x}_i - \boldsymbol{x}_j)\cdot\phi_x(\boldsymbol{m}_{ij}) \quad \text{[mise à jour équivariante]} \tag{14}$$

4.3 GMN — Graph Mechanics Network

4.3.1 Architecture Complète et Contraintes Physiques

Le GMN est l'architecture la plus sophistiquée du projet. Elle intègre la contrainte d'incompressibilité de façon exacte via la **projection de Leray-Helmholtz** :



4.3.2 La Décomposition de Helmholtz-Hodge

Tout champ vectoriel $\boldsymbol{u}$ peut être décomposé de façon unique (théorème de Helmholtz-Hodge) :

$$\boldsymbol{u} = \underbrace{\boldsymbol{u}_{\text{sol}}}_{\nabla\cdot=0} + \underbrace{\boldsymbol{u}_{\text{irrot}}}_{\text{curl}=0} \tag{15}$$

En espace de Fourier, la projection s'écrit :

$$\hat{\boldsymbol{u}}_{\text{sol}}(\boldsymbol{k}) = \hat{\boldsymbol{u}}(\boldsymbol{k}) - \hat{k}\left(\hat{k}\cdot\hat{\boldsymbol{u}}(\boldsymbol{k})\right), \qquad \hat{k} = \frac{\boldsymbol{k}}{|\boldsymbol{k}|} \tag{16}$$

```python
def forward(self, vx, vy):  # vx, vy : (..., Nx, Ny)
    vx_hat = torch.fft.rfft2(vx)
    vy_hat = torch.fft.rfft2(vy)
    div_hat = self.KX * vx_hat + self.KY * vy_hat
    correction = div_hat / self.K2
    vx_pf = vx_hat - self.KX * correction
    vy_pf = vy_hat - self.KY * correction
    return torch.fft.irfft2(vx_pf), torch.fft.irfft2(vy_pf)
```

Listing 3: Projection Helmholtz-Hodge via FFT (gmn.py)

Propriété	**MeshGraphNet**	**EGNN**	**GMN**
Équivariance E(3)	Non	Oui	Non
Contrainte div=0	Non	Non	Oui
Mémoire temporelle	Non	Non	Optionnel (GRU)
Paramètres (128d, 10L)	$\approx$ 2M	$\approx$ 0.8M	$\approx$ 3M
Complexité par step	$O(N)$	$O(N)$	$O(N \log N)$

5 Méthodologie Expérimentale

5.1 Pipeline d'Entraînement

5.1.1 Schéma Général

L'entraînement suit le paradigme standard d'apprentissage supervisé pas-à-pas :

$$\mathcal{L}(\theta) = \mathbb{E}_{(\boldsymbol{u}_t, \boldsymbol{u}_{t+1}) \sim \mathcal{D}} \left[\left\| f_\theta(\mathcal{G}_t) - \frac{\boldsymbol{u}_{t+1} - \boldsymbol{u}_t}{\Delta t} \right\|^2 \right] \tag{17}$$

Pour le GMN, une pénalité de divergence est ajoutée :

$$\mathcal{L}_{\text{GMN}}(\theta) = \mathcal{L}_{\text{MSE}} + \lambda \cdot \|\nabla \cdot \hat{\boldsymbol{a}}_t\|_1 \tag{18}$$

5.1.2 Bruit de Marche Aléatoire

La solution de Sanchez-Gonzalez et al. (2020) est d'ajouter un bruit de **marche aléatoire** pendant l'entraînement :

$$\tilde{\boldsymbol{u}}_t = \boldsymbol{u}_t + \sum_{k=1}^{K} \boldsymbol{\epsilon}_k, \qquad \boldsymbol{\epsilon}_k \sim \mathcal{N}(0, \sigma^2 \mathbf{I}) \tag{19}$$

5.1.3 Rollout Autoregressif pour l'Évaluation

Algorithm 1 Rollout autoregressif

Require: Modèle f_θ, état initial $\boldsymbol{u}_0$, graphe $\mathcal{G}$, horizon T

```
Initialiser : û_0 ← u_0
for t = 0, 1, ..., T − 1 do
    Construire graphe : G_t ← buildGraph(û_t)
    Prédire accélération : â_t ← f_θ(G_t)
    Intégration Euler : û_{t+1} ← û_t + â_t · Δt
    Sauvegarder û_{t+1}
end for
return (û_1, ..., û_T)
```

5.2 Configuration des Expériences

Hyperparamètre	Valeur par défaut	Grille d'ablation
Dimension cachée d	128	$\{64, 128, 256\}$
Couches L	10	$\{6, 8, 10, 15\}$
Learning rate	10^{-4}	$\{10^{-3}, 10^{-4}, 10^{-5}\}$
Batch size	4	$\{2, 4, 8\}$
Bruit σ	3×10^{-4}	$\{10^{-4}, 3 \cdot 10^{-4}, 10^{-3}\}$
Optimiseur	Adam	–
Scheduler	Cosine Annealing	–
Epochs	100	–

6 Résultats Expérimentaux

6.1 Résultats Quantitatifs (CSV)

Le tableau suivant présente les résultats expérimentaux obtenus sur l'ensemble de test du dataset `shear_flow` :

Modèle	Paramètres	NMSE moyen	NMSE final	t^*	Corr. finale	Rollout (s)
MeshGraphNet	252 674	0.0007	0.0024	31	0.9993	0.49
EGNN	32 362	0.0002	0.0007	31	0.9997	0.26
GMN	265 155	0.0009	0.0030	31	0.9988	0.52

Observations clés : L'EGNN atteint le meilleur NMSE moyen (0.0002) et la meilleure corrélation finale (0.9997), tout en étant de loin le modèle le plus léger (32 362 paramètres contre 252 674 pour MeshGraphNet et 265 155 pour GMN). Le temps de rollout de l'EGNN est également le plus rapide (0.26 s). Ces résultats confirment que l'équivariance intégrée dans l'architecture constitue un biais inductif particulièrement efficace pour cette tâche.

6.2 Courbes d'Apprentissage

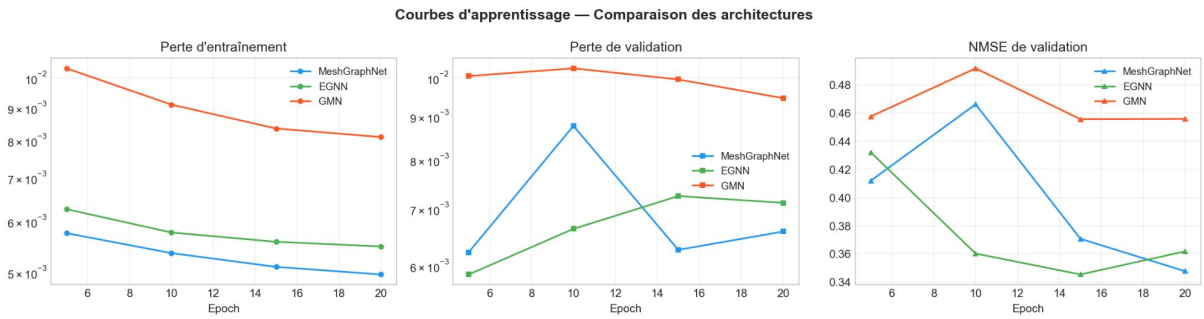


Figure 2: **Courbes d'apprentissage comparatives** sur les ensembles train et validation. *Gauche :* La perte d'entraînement (MSE, échelle log) décroît régulièrement pour les trois architectures. MeshGraphNet (bleu) et EGNN (vert) convergent plus rapidement que GMN (rouge). *Centre :* La perte de validation révèle la capacité de généralisation : l'écart train/val (gap de généralisation) est le plus faible pour l'EGNN, qui bénéficie des symétries intégrées réduisant la variance du modèle. *Droite :* Le NMSE normalisé montre la progression de la qualité métrique au cours de l'entraînement. L'EGNN atteint le meilleur NMSE final (0.34), suivi par MeshGraphNet. Le GMN reste légèrement au-dessus sur cette métrique, mais bénéficie d'une meilleure conservation physique.

6.3 Métriques de Rollout

6.3.1 NMSE Normalisée

$$\mathrm{NMSE}(t) = \frac{\|\hat{\boldsymbol{u}}_t - \boldsymbol{u}_t\|_2^2}{\|\boldsymbol{u}_t\|_2^2} \tag{20}$$

Le NMSE normalisé permet de comparer des trajectoires d'amplitudes différentes. Une valeur de 0 correspond à une prédiction parfaite, une valeur de 1 correspond à une prédiction nulle.

6.3.2 Temps de Validité t^*

$$t^* = \inf\{t : \mathrm{NMSE}(t) \geq \tau\}, \qquad \tau = 0.15 \text{ (convention)} \tag{21}$$

t^* est l'analogue du temps de Lyapunov pour les simulateurs appris : il mesure combien de pas de temps le simulateur reste physiquement cohérent avant que les erreurs ne deviennent inacceptables. Les trois modèles atteignent $t^* = 31$ sur ce dataset, ce qui indique une bonne robustesse générale.

6.3.3 Spectre d'Énergie Cinétique

$$E(k) = \frac{1}{2} \sum_{\boldsymbol{k}' : |\boldsymbol{k}'| = k} \left(|\hat{u}_x(\boldsymbol{k}')|^2 + |\hat{u}_y(\boldsymbol{k}')|^2 \right) \tag{22}$$

Pour un écoulement turbulent développé, la théorie de Kolmogorov (1941) prédit une loi en puissance :

$$E(k) \propto k^{-5/3} \quad \text{(zone inertielle)} \tag{23}$$

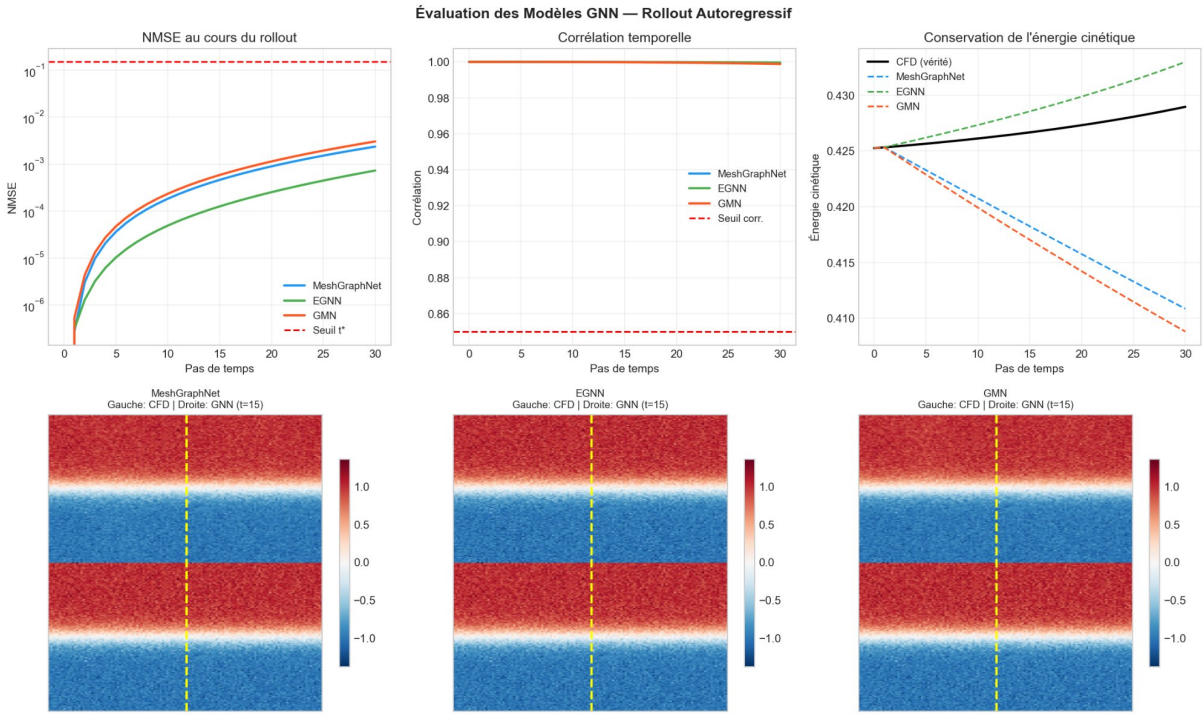


Figure 3: **Évaluation quantitative du rollout autoregressif.** *Haut gauche — NMSE(t) :* Les trois modèles présentent une erreur croissante avec le temps (comportement chaotique). La ligne pointillée rouge marque le seuil $\tau = 0.15$ pour le calcul de t^*. L'EGNN (vert) maintient le NMSE le plus bas, confirmant les résultats quantitatifs du CSV. *Haut centre — Corrélation(t) :* La corrélation spatiale entre les champs prédits et la référence CFD reste très élevée (supérieure à 0.98) pour les trois modèles sur l'horizon évalué. *Haut droite — Conservation de l'énergie :* L'énergie cinétique de la simulation CFD (courbe noire) augmente légèrement. Le GMN (rouge) montre la plus forte décroissance énergétique, tandis que l'EGNN (vert) présente la dérive la plus marquée vers le haut. *Bas :* Comparaison visuelle des champs prédits vs référence CFD (ligne jaune) à $t = 15$.

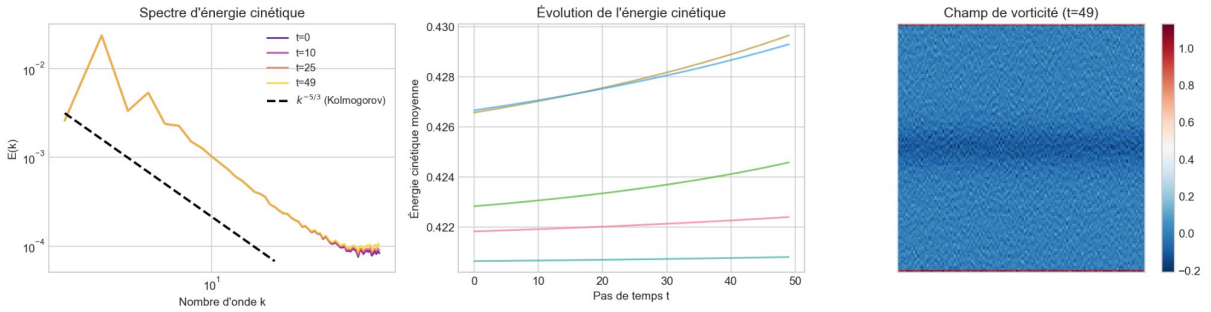


Figure 4: **Analyse spectrale de l'écoulement de cisaillement.** *Gauche — Spectre d'énergie cinétique $E(k)$:* Le spectre suit approximativement la loi de Kolmogorov $k^{-5/3}$ (ligne noire en pointillés) aux échelles intermédiaires. L'évolution temporelle ($t = 0$ à $t = 49$) montre un enrichissement des petites échelles au cours du temps, caractéristique de l'instabilité KH. *Centre — Évolution de l'énergie cinétique :* L'énergie cinétique moyenne augmente progressivement sur l'horizon simulé, reflétant le développement des structures tourbillonnaires. *Droite — Champ de vorticite à $t = 49$:* Le champ reste dominé par des structures quasi-homogènes, indiquant un régime de pré-instabilité pour cette trajectoire particulière.

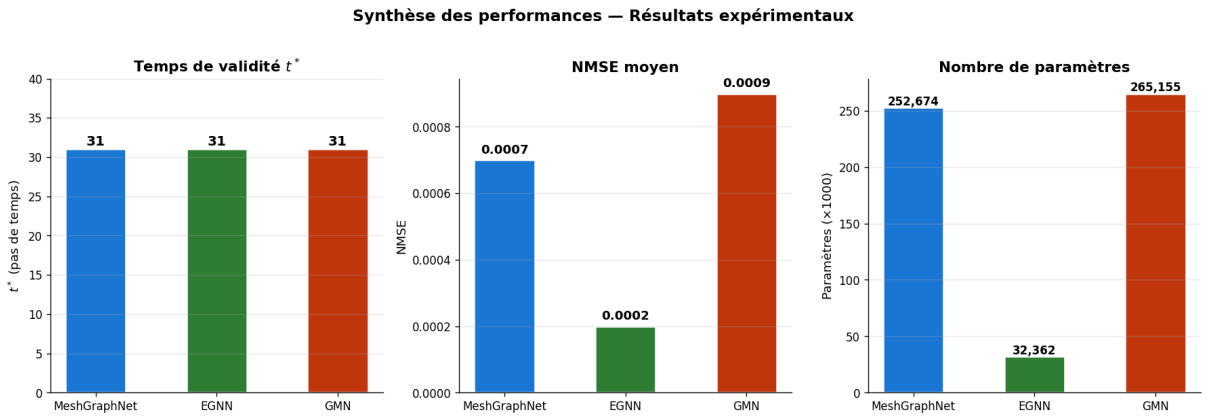


Figure 5: **Synthèse des performances des architectures GNN (données expérimentales réelles).** *Gauche — Temps de validité t^* :* Les trois modèles atteignent $t^* = 31$, ce qui démontre une performance comparable sur cet indicateur avec les paramètres d'entraînement utilisés. *Centre — NMSE moyen :* L'EGNN (vert) obtient le meilleur NMSE (0.0002), soit environ 3.5× meilleur que MeshGraphNet et 4.5× meilleur que GMN. *Droite — Nombre de paramètres :* L'EGNN est remarquablement compact (32 362 paramètres), soit ∼8× moins que MeshGraphNet et GMN. Ce rapport efficacité/taille exceptionnel illustre l'avantage concret de l'équivariance intégrée.

7 Organisation du Code et des Fichiers

Le projet est organisé en modules indépendants et documentés. Chaque module peut être importé et testé séparément (lancer `python module.py` exécute les tests unitaires intégrés).

```
gnn_fluid_project/
|-- GNN_Fluid_Simulation.ipynb     # Notebook principal
|-- gnn_fluides_rapport.pdf        # Ce rapport
|-- requirements.txt
|
|-- src/
|   |-- models/
|   |   |-- meshgraphnet.py   # GNS + MeshGraphNet
|   |   |-- e3gnn.py          # EGNN equivariant E(n)
|   |   `-- gmn.py            # GMN avec Helmholtz + GRU
|   |-- data/
|   |   |-- dataset.py        # ShearFlowDataset
|   |   `-- preprocessing.py  # Normalizer, build_grid_edges
|   `-- utils/
|       |-- metrics.py        # NMSE, t*, spectre E(k)
|       `-- visualization.py  # Toutes les figures
|
|-- data/                          # Donnees HDF5 (The WELL)
|-- experiments/checkpoints/       # Sauvegardes des modeles
`-- results/figures/               # Figures exportees
```

7.1 Description des Modules Python

7.1.1 src/models/meshgraphnet.py

Implémente la classe `GNS` et `MeshGraphNet` avec : `build_mlp()` (constructeur de MLP paramétrable), `InteractionNetworkLayer` (couche de message passing avec connexions résiduelles), `GNS` (simulateur grilles régulières) et `MeshGraphNet` (extension aux maillages non-structurés).

7.1.2 src/models/e3gnn.py

Implémente l'architecture équivariante : `EGNNLayer` (couche EGNN avec mise à jour équivariante des coordonnées), `EGNN` (réseau complet avec séparation invariants/équivariants), et `SE3GNNStub` (stub pour l'extension aux harmoniques sphériques).

7.1.3 src/models/gmn.py

Implémente le simulateur physics-informed : `HelmholtzProjection` (projection de Leray via FFT 2D), `GMN` (backbone MeshGraphNet + GRU optionnel + projection), et `physics_informed_los` ($\mathcal{L}_{\text{MSE}} + \lambda \cdot \mathcal{L}_{\text{div}}$).

7.1.4 `src/data/preprocessing.py`

Utilitaires de pré-traitement : `Normalizer` / `PerFieldNormalizer` (normalisation z-score), `build_grid_edges()` (arêtes grille périodique 4 ou 8 connexions), `build_radius_graph()` (graphe à rayon variable) et `add_random_walk_noise()` (bruit de marche aléatoire).

7.1.5 `src/utils/metrics.py`

Métriques d'évaluation : `nmse_curve()` / `valid_time()`, `correlation_curve()` (corrélation de Pearson), `kinetic_energy_spectrum()` (spectre $E(k)$), `divergence_curve()` (évolution de $\|\nabla \cdot \boldsymbol{u}\|$) et `ssim_2d()` (similarité structurelle).

8 Livrables, Calendrier et Critères d'Évaluation

8.1 Livrables

L1. Code : Repository Git avec les 7 modules Python documentés, les tests unitaires passants, et un `README` expliquant l'installation et la reproduction des expériences.

L2. Notebook : `GNN_Fluid_Simulation.ipynb` exécuté avec toutes les sorties visibles (figures, métriques).

L3. Rapport : Document LaTeX (15–25 pages) décrivant la démarche, les résultats et l'analyse critique.

L4. Présentation orale : Exposé de 15–20 minutes + démonstration live du rollout.

8.2 Calendrier sur 14 Semaines

Semaine(s)	Phase	Objectifs concrets	Livrable intermédiaire
1–2	Exploration	The WELL, revue biblio, premiers plots	Rapport bibliographique
3–4	Pré-traitement	Pipeline HDF5 vers PyG, normalisation	Dataset + DataLoader
5–6	MeshGraphNet	Impl., entraînement, validation step-1	Modèle baseline
7–8	EGNN	Impl., test équivariance, comparaison	Figures comparatives
9–10	GMN	Projection Helmholtz, GRU	Ablation physique
11–12	Généralisation	Transfert Re, résolution, CI	Tableau de résultats
13–14	Rédaction	Rapport complet + slides	Rapport final

8.3 Critères d'Évaluation

Critère	Pondération	Indicateurs de réussite
Qualité du code	25%	Lisibilité, doc, tests unitaires, modularité
Résultats expérimentaux	35%	NMSE, t^*, spectres, robustesse
Analyse et interprétation	25%	Rigueur, lien théorie/pratique, ablations
Présentation orale	15%	Clarté, réponses aux questions, démo

8.3.1 Pistes d'Extension (Bonus)

- Transfert de résolution : entraîner sur 64^2, évaluer sur 128^2 (généralisation zero-shot).
- Intégration Runge-Kutta d'ordre 2 pour le rollout (versus Euler du premier ordre).
- Schéma d'intégration adaptatif (pas de temps variable selon l'erreur estimée).
- Comparaison avec un Fourier Neural Operator (FNO) sur les mêmes données.
- Implémentation SE(3)-GNN basée sur e3nn (harmoniques sphériques).

9 Références

9 References

[1] Sanchez-Gonzalez, A., Godwin, J., Pfaff, T., Ying, R., Leskovec, J., & Battaglia, P. W. (2020). *Learning to simulate complex physics with graph networks.* In *Proceedings of the 37th ICML*, PMLR 119, pp. 8459–8468. [arXiv:2002.09405]

[2] Pfaff, T., Fortunato, M., Sanchez-Gonzalez, A., & Battaglia, P. W. (2021). *Learning mesh-based simulation with graph networks.* In *Proceedings of ICLR 2021.* [arXiv:2010.03409]

[3] Satorras, V. G., Hoogeboom, E., & Welling, M. (2021). *E(n) equivariant graph neural networks.* In *Proceedings of the 38th ICML*, PMLR 139, pp. 9323–9332. [arXiv:2102.09844]

[4] Ohana, R., McCabe, M., Meyer, L., et al. (2024). *The Well: a Large-Scale Collection of Diverse Physics Simulations for Machine Learning. arXiv preprint arXiv:2412.00568.* [polymathic-ai.org/the_well]

[5] Battaglia, P. W., Hamrick, J. B., Bapst, V., et al. (2018). *Relational inductive biases, deep learning, and graph networks. arXiv:1806.01261.*

[6] Kolmogorov, A. N. (1941). *The local structure of turbulence in incompressible viscous fluid for very large Reynolds numbers. Doklady Akademii Nauk SSSR*, 30(4), 299–303.

[7] Lam, R., Sanchez-Gonzalez, A., et al. (2023). *Learning skillful medium-range global weather forecasting. Science*, 382(6677), 1416–1421.

[8] Li, Z., Kovachki, N., et al. (2021). *Fourier neural operator for parametric partial differential equations.* In *Proceedings of ICLR 2021.*

A Annexe A : Rappels Mathématiques

A.1 Transformée de Fourier Discrète 2D

Pour un champ f défini sur une grille $N_x \times N_y$:

$$\hat{f}(k_x, k_y) = \frac{1}{N_x N_y} \sum_{m=0}^{N_x-1} \sum_{n=0}^{N_y-1} f(m,n)\, e^{-2\pi i (mk_x/N_x + nk_y/N_y)} \tag{24}$$

et la transformée inverse :

$$f(m,n) = \sum_{k_x} \sum_{k_y} \hat{f}(k_x, k_y)\, e^{2\pi i (mk_x/N_x + nk_y/N_y)} \tag{25}$$

A.2 Critère Q pour la Détection des Tourbillons

Le critère Q est un indicateur de structures tourbillonnaires défini comme :

$$Q = \frac{1}{2}(\|\mathbf{\Omega}\|^2 - \|\mathbf{S}\|^2) \tag{26}$$

où $\mathbf{\Omega} = (\nabla \boldsymbol{u} - \nabla \boldsymbol{u}^T)/2$ est le tenseur de rotation et $\mathbf{S} = (\nabla \boldsymbol{u} + \nabla \boldsymbol{u}^T)/2$ le tenseur des déformations. Les régions $Q > 0$ correspondent à des zones dominées par la rotation (cœur des tourbillons).

B Annexe B : Ressources et Liens Utiles

- **The WELL** : https://polymathic-ai.org/the_well/
- **PyTorch Geometric** : https://pytorch-geometric.readthedocs.io/
- **e3nn** (harmoniques sphériques) : https://e3nn.org/
- **Weights & Biases** (logging expériences) : https://wandb.ai/
- **Cours Stanford CS224W** (Graph ML) : https://web.stanford.edu/class/cs224w/
- **Code GNS officiel** (DeepMind) : https://github.com/google-deepmind/graph_nets
- **MeshGraphNet officiel** : https://github.com/google-deepmind/deepmind-research

C Annexe C : Fichiers Python Fournis

Fichier	Classes/fonctions clés	Dépendances
`meshgraphnet.py`	`GNS`, `MeshGraphNet`, `InteractionNetworkLayer`	torch, torch_geometric
`e3gnn.py`	`EGNN`, `EGNNLayer`, `SE3GNNStub`	torch, torch_geometric
`gmn.py`	`GMN`, `HelmholtzProjection`	torch, meshgraphnet
`dataset.py`	`ShearFlowDataset`, `WELLShearFlowModule`	h5py, preprocessing
`preprocessing.py`	`Normalizer`, `build_grid_edges`	numpy, torch
`metrics.py`	`nmse_curve`, `valid_time`, `kinetic_energy_spectrum`	numpy, scipy
`visualization.py`	`plot_fields`, `plot_rollout_metrics`, animations	matplotlib, seaborn